

INTERNSHIP REPORT

Task 3: Exploratory Data Analysis (EDA)

AI & ML Internship Program

Dataset Used: Iris Dataset

Tools Used: Python, Pandas, Matplotlib, Seaborn, Google Colab

Submitted By:

MODI DHRUV BHAVESHBHAI

TABLE OF CONTENTS

- 1. Introduction**
- 2. Objective of the Task**
- 3. Dataset Information**
- 4. Libraries Used**
- 5. Data Preprocessing**
- 6. Univariate Analysis**
 - A) Histogram Analysis**
 - B) Boxplot Analysis**
- 7. Bivariate Analysis**
 - A) Scatter Plot Analysis**
 - B) Bar Plot Analysis**
- 8. Correlation Analysis**
- 9. Pairplot Analysis**
- 10. Final Insights**
- 11. Conclusion**
- 12. References**

1. INTRODUCTION

What is Data Analysis?

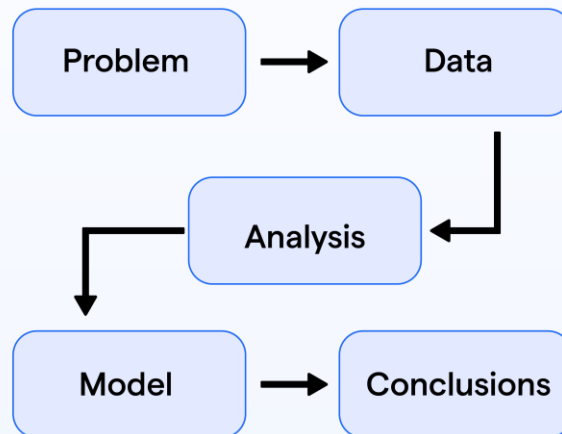
Data Analysis is the systematic process of cleaning, transforming, and modeling data to discover useful information for business decision-making. In today's digital world, massive amounts of data are generated every second. However, raw data on its own is meaningless. Data analysis acts as the bridge that turns this raw, unstructured data into actionable insights, helping individuals and organizations understand trends, evaluate performance, and predict future outcomes.

What is Exploratory Data Analysis (EDA)?

Exploratory Data Analysis (EDA) is an approach to analyzing datasets to summarize their main characteristics, often employing statistical graphics and other data visualization methods. Promoted by American statistician John Tukey, EDA encourages data scientists to explore the data, formulate hypotheses, and assess assumptions before applying any formal statistical models.

Unlike traditional hypothesis testing, EDA is akin to a detective's work. It does not start with a rigid assumption; instead, it lets the data speak for itself. By using various graphical techniques (like histograms, scatter plots, and box plots) and quantitative methods (like summary statistics), EDA helps uncover hidden structures within the dataset.

Exploratory Data Analysis



Importance of EDA in Data Science and Machine Learning

In the pipeline of Machine Learning and Data Science, EDA is arguably the most critical step. If a machine learning model is fed with flawed, noisy, or completely un-scaled data, the resulting predictions will be highly inaccurate—a concept known in computer science as 'Garbage In, Garbage Out' (GIGO). EDA prevents this by allowing the data scientist to intimately understand the dataset's nuances.

By taking the time to conduct EDA, we can determine the optimal features to use for our models, identify whether we need to collect more data, and select the correct machine learning algorithms suited for the data's specific distribution.

Why Visualization is Important

Human brains are intrinsically wired to process visual information much faster than raw numbers or text. When staring at a spreadsheet with thousands of rows and columns, it is nearly impossible to spot a trend. However, plotting that same data onto a simple graph instantly reveals patterns, peaks, valleys, and anomalies.

Visualization simplifies complexity. It serves as a universal language that allows data professionals to communicate their findings to non-technical stakeholders effectively. In Python, libraries like Matplotlib and Seaborn provide powerful, intuitive tools to create these vital visual summaries.

How EDA Helps Before Machine Learning Model Building

Before a single line of machine learning predictive code is written, EDA lays the foundation through four main pillars:

1. **Pattern Detection:** EDA helps us see the overall shape of the data. Does it follow a normal distribution? Are there seasonal trends? Understanding patterns dictates how we approach feature engineering and model selection.

2. **Outlier Detection:** Outliers are data points that differ significantly from other observations. They can be caused by measurement errors or they might indicate a novel phenomenon. Machine learning algorithms, particularly linear regression, are highly sensitive to outliers. EDA helps us spot them (usually via boxplots) so we can decide whether to remove or transform them.

3. Data Understanding: Through EDA, we learn the basic characteristics of our dataset: the types of variables (categorical, numerical), the presence of missing values, and the scale of the measurements. This understanding is vital for proper data preprocessing.

4. Relationship Analysis: By visualizing how two or more variables interact (using scatter plots or heatmaps), we can identify correlations. If two features are highly correlated, they provide redundant information, and we might choose to drop one to simplify our machine learning model without losing accuracy.

2. OBJECTIVE OF THE TASK

The primary objectives of performing this Exploratory Data Analysis on the Iris dataset are:

- • Understanding Dataset Structure: To load the dataset and comprehend its dimensions, variable types, and summary statistics.
- • Performing Visualization: To utilize Python libraries to generate meaningful graphs that represent the data visually.
- • Detecting Patterns: To analyze the distribution of various flower measurements and observe how they group together.
- • Finding Relationships: To explore how different features (like petal length and width) relate to each other.
- • Detecting Outliers: To identify any anomalous data points that deviate from the normal distribution of the features.
- • Understanding Correlation: To compute and visualize the correlation matrix to understand the strength of the linear relationships between the variables.

By achieving these objectives, the final goal is to prepare a clean, well-understood dataset that is perfectly suited for training accurate classification machine learning models in the future.

3. DATASET INFORMATION

Dataset Name: Iris Dataset

The Iris dataset is perhaps the most famous and widely used beginner dataset in the field of Machine Learning and Statistics. It was introduced by the British statistician and biologist Ronald Fisher in his 1936 paper. The dataset consists of 150 samples from each of three species of Iris flowers (Setosa, Versicolor, and Virginica). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters.

Why is it famous? It is famous because it is simple, requires no complex preprocessing, has no missing values, and the classes are relatively easy to separate linearly (especially Setosa from the other two). It serves as the perfect 'Hello World' for classification algorithms.

Total Rows: 150

Total Columns: 5

Column Descriptions

Column Name	Data Type	Description
sepal_length	Float	Length of the flower sepal in cm
sepal_width	Float	Width of the flower sepal in cm
petal_length	Float	Length of the flower petal in cm
petal_width	Float	Width of the flower petal in cm
species	String/Object	Target class: setosa, versicolor, virginica

4. LIBRARIES USED

For this EDA task, several core Python libraries were utilized to process and visualize the data effectively:

Pandas: Pandas is an open-source data manipulation and analysis tool built on top of the Python programming language. It provides the 'DataFrame' object, which allows us to read the CSV file, handle missing data, and perform statistical operations easily. In this project, Pandas was used to load the Iris dataset and generate statistical summaries using functions like `describe()` and `info()`.

Matplotlib: Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It acts as the backbone for plotting in this task. We used Matplotlib's `pyplot` module to structure our figures, define axes, and plot basic charts like histograms and scatter plots.

Seaborn: Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. Seaborn was heavily used in this project to create visually appealing Boxplots, Bar Plots, Correlation Heatmaps, and the comprehensive Pairplot, utilizing its built-in color palettes and statistical aggregation features.

NumPy (Optional/Background): While not called explicitly for complex calculations, NumPy operates silently in the background alongside Pandas and Seaborn to handle array computations and linear algebra operations rapidly.

Google Colab: Google Colaboratory is a hosted Jupyter notebook service that requires no setup to use and provides free access to computing resources. It was the primary environment used to write and execute the Python code for this EDA task, offering seamless inline visualization of our generated graphs.

5. DATA PREPROCESSING

Data Preprocessing is the initial step where we inspect the raw data to ensure it is clean and formatted correctly before visualization. Here is the step-by-step process followed:

Step 1: Loading the Dataset

We imported the pandas library and read the CSV file into a DataFrame using `pd.read_csv()`.

Step 2: Viewing First Rows (`head()`)

Using `df.head()`, we displayed the first five rows of the dataset. This gives us a quick snapshot of the data format, showing the numerical columns and the target text column (species).

Step 3: Checking Dataset Information (`info()`)

The `df.info()` function was used to print a concise summary of the DataFrame. It revealed that there are 150 total entries, 4 float64 columns, and 1 object column. It also showed that memory usage is extremely light.

Step 4: Checking Statistical Summary (`describe()`)

By calling `df.describe()`, we generated descriptive statistics. This provided the mean, standard deviation, minimum, maximum, and quartile values for sepal and petal dimensions. It instantly showed us the scale of our data without needing any plots.

Step 5: Checking Missing Values (`isnull().sum()`)

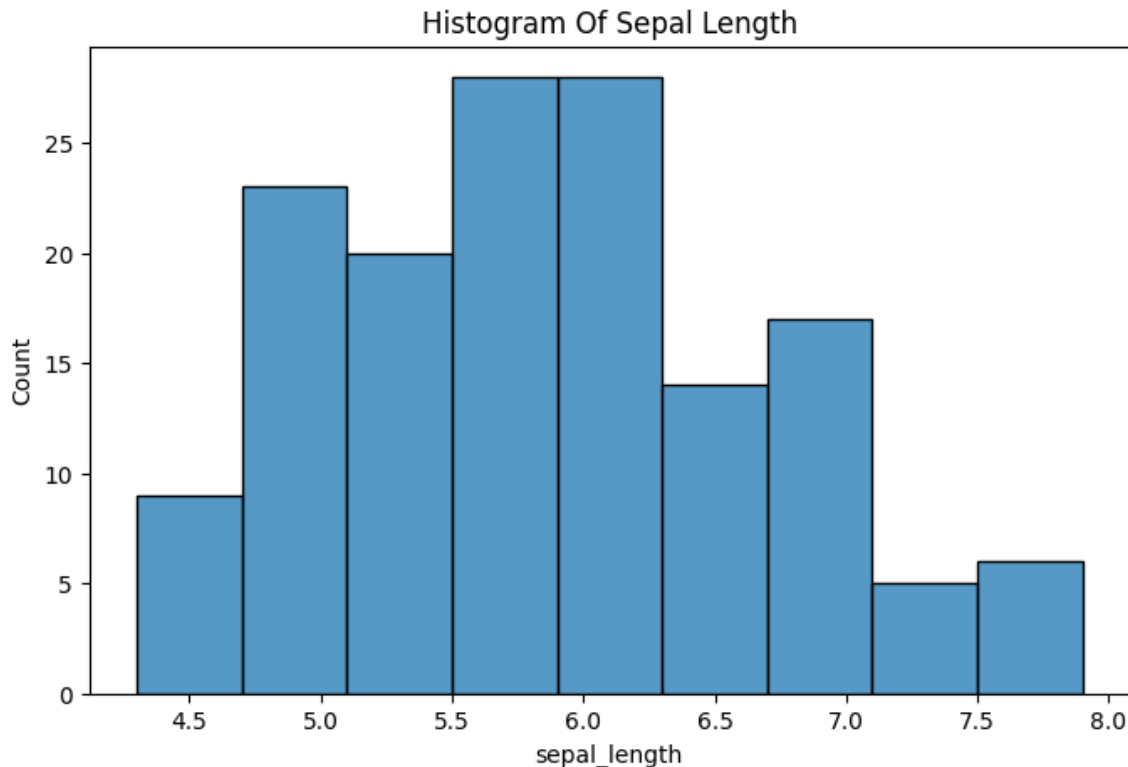
We used `df.isnull().sum()` to check if there were any missing (NaN) values in our columns.

Result: No missing values were found in the dataset. This means the Iris dataset is complete, and we do not need to perform missing value imputation (like filling with mean or median).

6. UNIVARIATE ANALYSIS

Univariate analysis is the simplest form of analyzing data where we examine one variable (one feature) at a time. The purpose is to describe the data and find patterns within that specific column.

A) HISTOGRAM ANALYSIS



What is a Histogram?

A histogram is a graphical representation that organizes a group of data points into user-specified ranges (bins). It looks similar to a bar chart but groups continuous data into ranges to show frequency distribution.

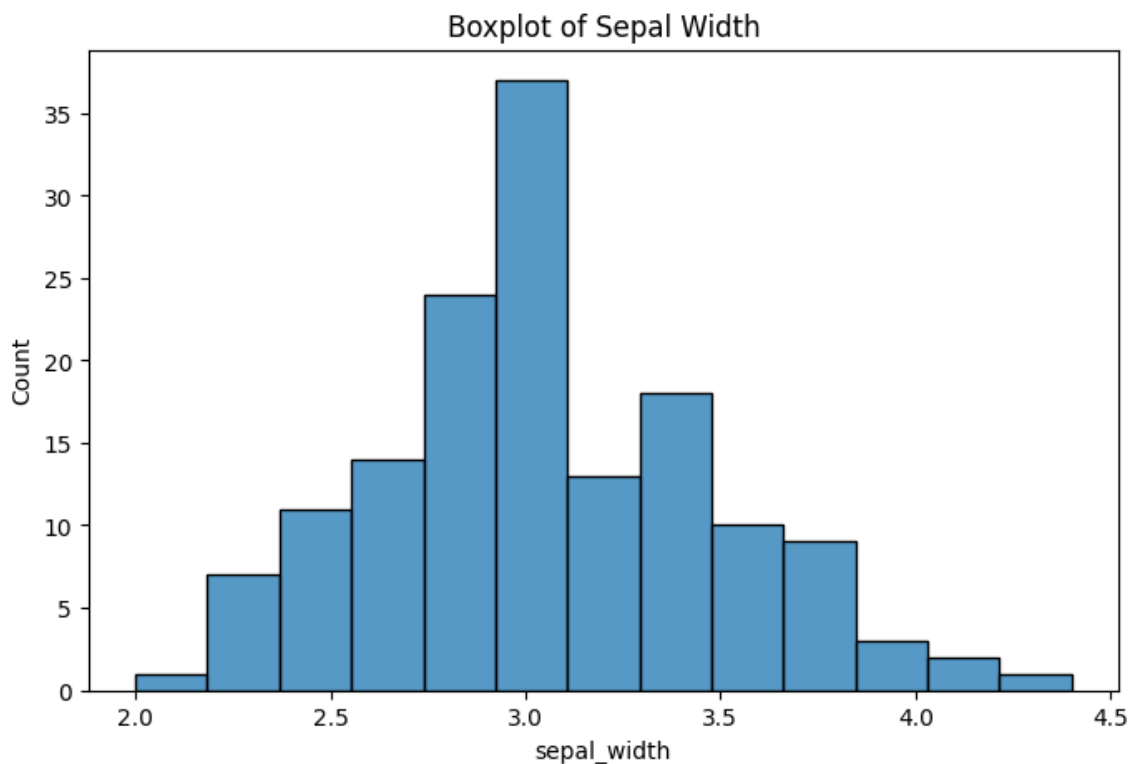
Why is it used?

It is used to understand the probability distribution of a continuous variable. It helps us see where most values are clustered and whether the data is skewed to the left or right.

Interpretation & Observation:

Looking at the generated histogram for Sepal Length, we can observe the spread of the data. Most of the Iris flowers in this dataset possess a sepal length between 5.0 cm and 6.0 cm. The distribution is slightly somewhat close to a normal distribution, peaking around the 5.5 cm mark. Very few flowers have extremely short (below 4.5 cm) or extremely long (above 7.5 cm) sepal lengths.

B) BOXPLOT ANALYSIS



What is a Boxplot?

A boxplot (or box-and-whisker plot) is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median, third quartile (Q3), and maximum.

What are Outliers & How are they detected?

Outliers are data points that sit completely outside the expected distribution. In a boxplot, any data point outside the 'whiskers' (typically defined as 1.5 times the Interquartile Range, or IQR, above Q3 or below Q1) is plotted as an individual dot, making them visually unmistakable.

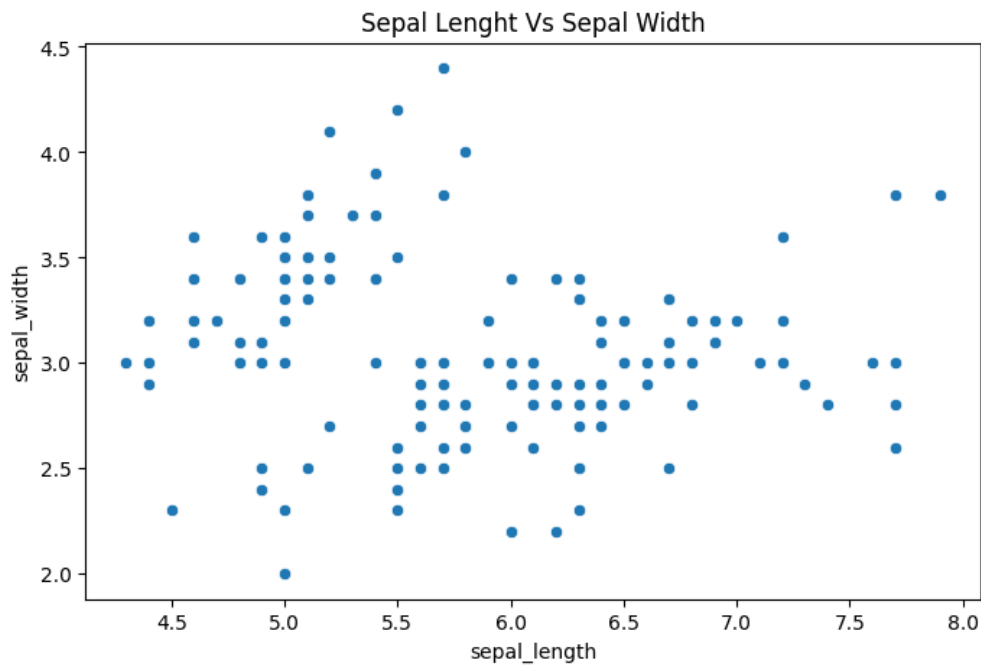
Interpretation & Observation:

The boxplot plotted for Sepal Width clearly displays the median line inside the central box. Crucially, we can observe a few isolated dots beyond the upper and lower whiskers. These are identified as outliers in the sepal width. While the majority of flowers have a sepal width tightly clustered between 2.8 and 3.3, a few anomalous flowers have widths dropping below 2.2 or extending above 4.0.

7. BIVARIATE ANALYSIS

Bivariate analysis involves the analysis of two variables to determine the empirical relationship between them.

A) SCATTER PLOT ANALYSIS



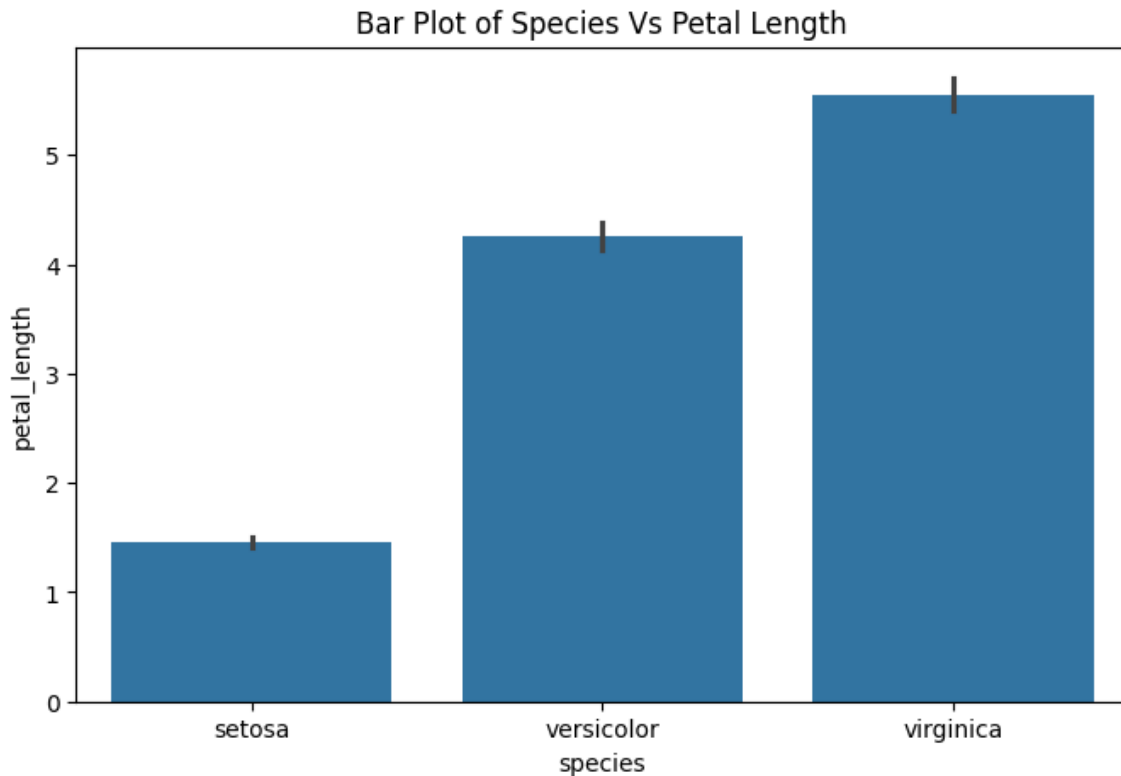
What is a Scatter Plot?

A scatter plot uses dots to represent values for two different numeric variables. The position of each dot on the horizontal and vertical axis indicates values for an individual data point.

Interpretation & Observation:

In the graph of Sepal Length (X-axis) versus Sepal Width (Y-axis), the data points are distributed across the grid. We can visually inspect the relationship. When color-coded by species, this scatter plot typically reveals that the 'Setosa' species clusters tightly together in the top left (shorter length, higher width), distinctly separating itself from the Versicolor and Virginica species which somewhat overlap.

B) BAR PLOT ANALYSIS

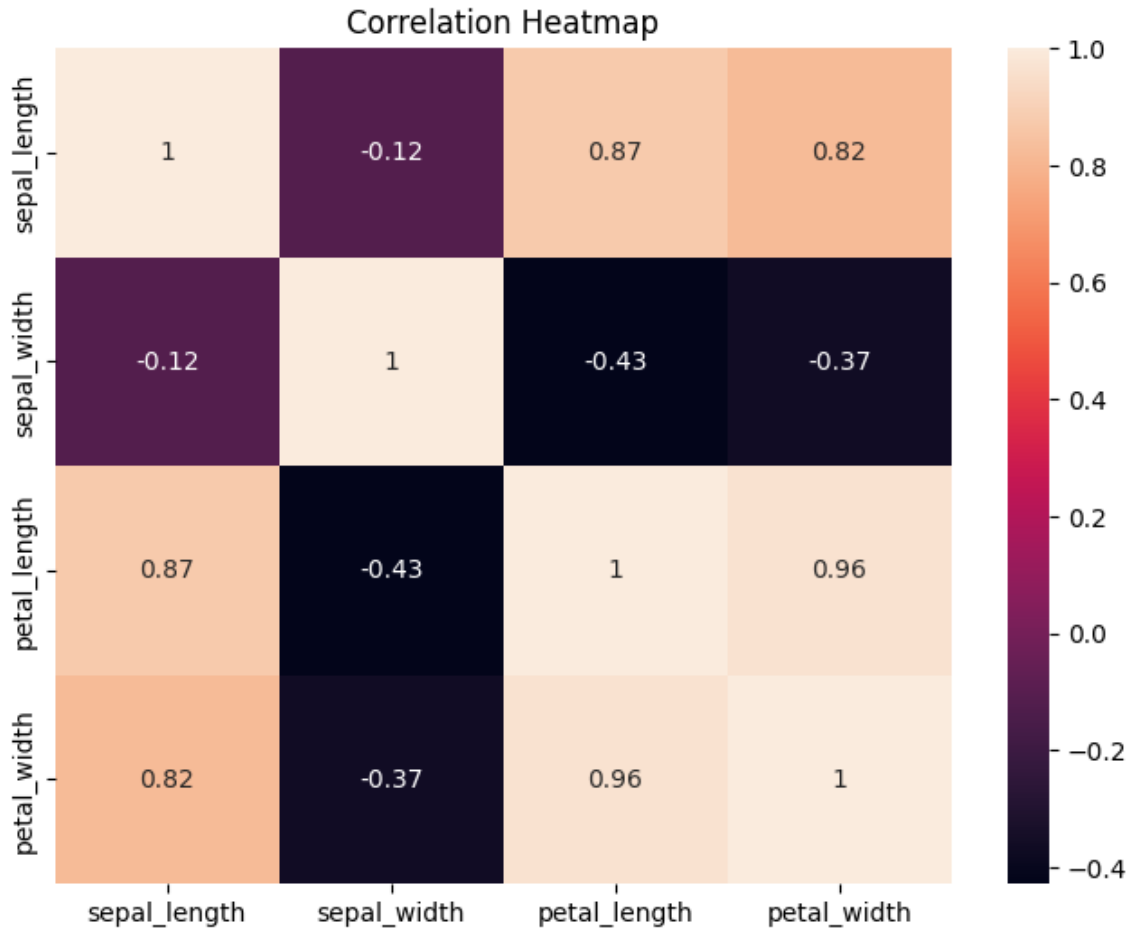


Observation and Analysis:

A bar plot categorizes categorical data against numerical data. By plotting Species on the X-axis and Petal Length on the Y-axis, we can instantly compare the average petal lengths across the three flower types.

The graph clearly illustrates that the Virginica species possesses the highest average petal length by a significant margin. Conversely, the Setosa species has exceptionally small petal lengths. This distinct stair-step pattern across the three species makes Petal Length an incredibly powerful feature for any classification machine learning model.

8. CORRELATION ANALYSIS



What is Correlation?

Correlation is a statistical measure that expresses the extent to which two variables are linearly related (meaning they change together at a constant rate).

- Positive Correlation: As one variable increases, the other also increases (values close to 1).
- Negative Correlation: As one variable increases, the other decreases (values close to -1).

The Heatmap Visualization:

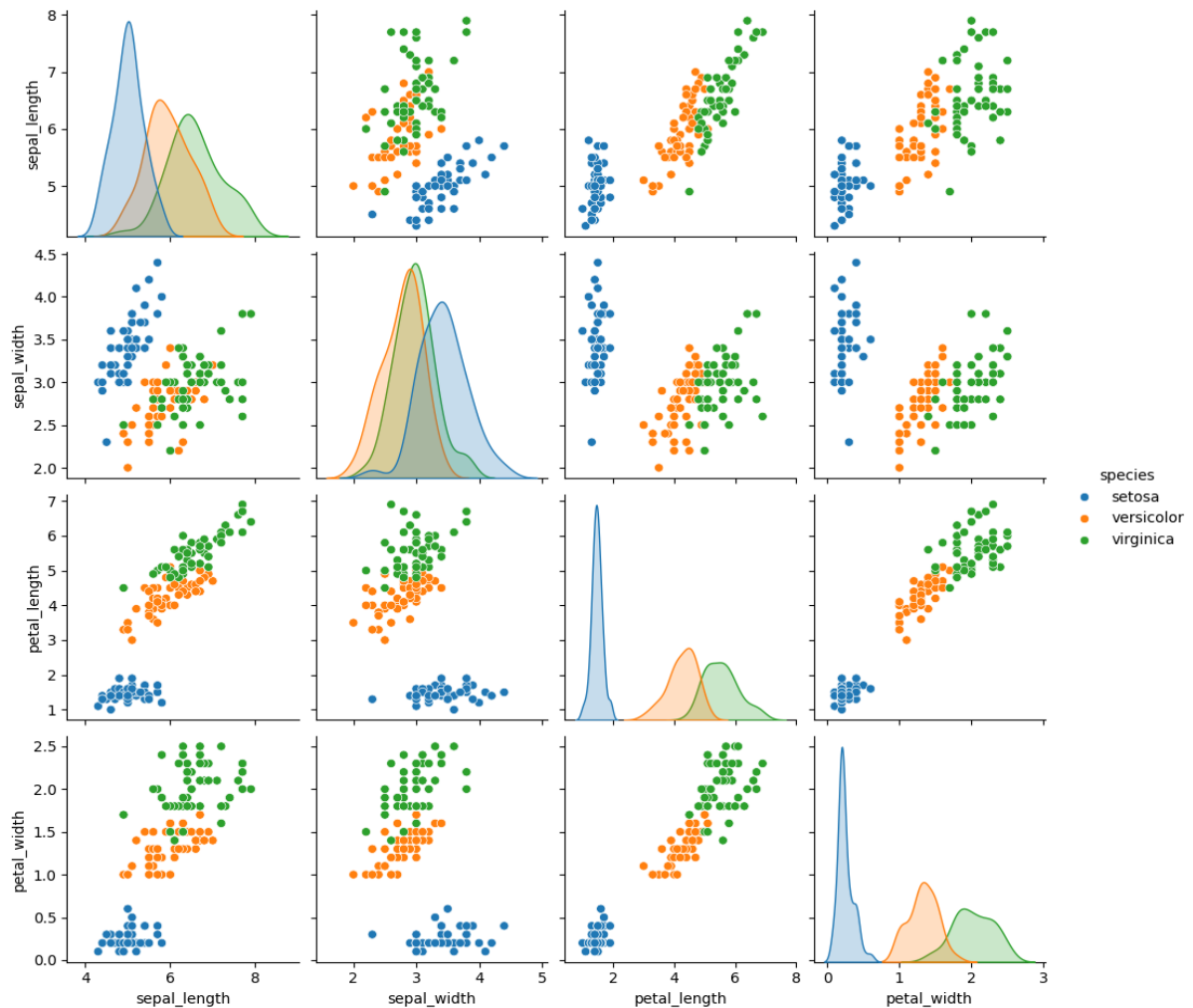
A correlation matrix displays the correlation coefficients between all combinations of variables. A heatmap uses color coding to make this matrix easily readable—darker or warmer colors usually indicate stronger relationships.

Detailed Interpretation:

1. Petal length and petal width show a very strong positive correlation (approximately 0.96). This means flowers with longer petals almost always have wider petals.
2. Sepal length is also strongly positively correlated with both petal length and petal width.
3. Interestingly, sepal width exhibits a weak negative correlation with sepal length and petal measurements.

Before modeling, this is vital. Since petal length and width are so highly correlated (0.96), they suffer from multicollinearity. In some models, we could drop one of these features to simplify the system without losing predictive power.

9. PAIRPLOT ANALYSIS



What is a Pairplot?

A pairplot allows us to plot pairwise relationships across an entire dataframe for all numerical columns simultaneously. It creates a grid of Axes such that each variable in the data is shared across the Y-axis across a single row and the X-axis across a single column. The diagonal axes usually display the univariate distribution (like a histogram or KDE) for the data in that column.

Why is it useful?

Instead of generating a dozen scatter plots manually, a single Seaborn pairplot function call generates a comprehensive 'bird's eye view' of every relationship in the dataset, color-coded by the target variable (species).

Observations:

- **Distinct Clustering:** Across almost all combinations of features, the Setosa species forms a distinct, tight cluster separated from the rest. This indicates that a simple linear model could easily classify Setosa flowers.
- **Petal Relationships:** The strong linear relationship between petal length and petal width is reaffirmed visually across the scatter plot grids.
- **Species Overlap:** Versicolor and Virginica show some overlap, particularly in sepal measurements, meaning they are slightly harder to distinguish from one another compared to Setosa.

10. FINAL INSIGHTS

After conducting a thorough Exploratory Data Analysis, the following core insights were derived from the Iris dataset:

1. Dataset Integrity: The dataset is remarkably clean, containing no missing or null values, making it highly robust for immediate processing.
2. High Correlation: Petal length and petal width are highly correlated (0.96), indicating they grow proportionally and serve as the strongest predictors for classification.
3. Species Distinctions: The Setosa species is clearly distinguishable and easily separable from the others based purely on its small petal dimensions and wide sepals.
4. Outlier Presence: A few outliers exist specifically within the sepal width feature. While not catastrophic, they should be noted for algorithms highly sensitive to outliers.
5. Comprehensive Overview: The Pairplot provided an excellent global understanding, confirming that while Versicolor and Virginica have slight overlaps, the features provided are sufficient for strong machine learning model performance.
6. Value of Visuals: Visualizations profoundly improved the understanding of the dataset, transforming raw numerical tables into immediately actionable biological trends.

11. CONCLUSION

In conclusion, this task successfully demonstrated the power and necessity of Exploratory Data Analysis (EDA). By utilizing Python, Pandas, Matplotlib, and Seaborn, we were able to unpack the famous Iris Dataset comprehensively.

The visualizations generated—from univariate histograms and boxplots to complex bivariate scatter plots and heatmaps—allowed us to fundamentally understand the underlying structure of the data. We identified clean relationships, detected minor outliers, and observed distinct patterns regarding the flower species.

Because the dataset is clean, free of missing values, and possesses strong correlations between key features, it is highly suitable for training machine learning models. This EDA phase ensures that subsequent predictive modeling steps will be built upon an accurate, well-understood foundation.

12. REFERENCES

- **Kaggle(Iris dataset):**
<https://www.kaggle.com/datasets/himanshunakrani/iris-dataset/data>
- **Google Colab:**
https://colab.research.google.com/drive/1Dzrh7EupG_FMEAyvNVK_hzGMj7YNIBifb?usp=sharing